[1]LAPORAN PRAKTIKUM PEKAN 4

PEMOGRAMAN BERORIENTASI OBJEK



Disusun oleh:

ANNAFI AL GHIFARI

2511533013

Dosen pengampu

Nurfiah, S.ST, M.Kom.

Asisten Praktikum

Dawi Syauqi

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

1

PADANG

2026

DAFTAR ISI

PROGRAM..... **Error! Bookmark not defined.**

KESIMPULAN..... 64

PERTEMUAN 1

1.1 Kode Program Rekening.java

```
package pekan1;

public class Rekening {

    String nomorRekening;
    String namaPemilik;
    double saldo;

    public Rekening(String nomor, String nama, double saldoAwal) {

        nomorRekening = nomor;
        namaPemilik = nama;
        saldo = saldoAwal;

        System.out.println(
                "Rekening atas nama " + namaPemilik +
                        " berhasil dibuat dengan saldo Rp" + saldo);
    }

    public void setorTunai(double nominal) {

        if (nominal >= 10000) {

            saldo += nominal;

            System.out.println(
                    "Setor tunai Rp" + nominal +
                            " berhasil. Saldo saat ini: Rp" + saldo);

        } else {

            System.out.println(
                    "Gagal, nominal setor minimal Rp10.000!");
        }
    }
```

```
public void tarikTunai(double nominal) {

    if (nominal < 10000) {

        System.out.println(
                "Gagal, nominal tarik tunai minimal Rp10.000!");

    } else if (nominal > saldo) {

        System.out.println(
                "Gagal, saldo tidak mencukupi, saldo Anda : Rp" + saldo);

    } else {

        saldo -= nominal;

        System.out.println(
                "Tarik tunai Rp" + nominal +
                        " berhasil. Saldo saat ini: Rp" + saldo);
    }
}

public void cekInformasi() {

    System.out.println("\n--- INFO REKENING ---");
    System.out.println("No. Rekening : " + nomorRekening);
    System.out.println("Nama Pemilik : " + namaPemilik);
    System.out.println("Saldo Akhir  : Rp" + saldo);
    System.out.println("---------------------");
}

public void gantiAkun(String nomor, String nama, double saldoAwal) {

    nomorRekening = nomor;
    namaPemilik = nama;
    saldo = saldoAwal;

    System.out.println(
            "Rekening atas nama " + namaPemilik +
```

```
                " berhasil diganti dengan saldo Rp" + saldo);
    }
}
```

1.1.1 Penjelasan

package pekan1; public class Rekening { Kode tersebut digunakan untuk menentukan bahwa class Rekening berada di dalam package pekan1. Class Rekening digunakan sebagai tempat untuk menyimpan data dan fungsi yang berkaitan dengan rekening bank.
String nomorRekening; String namaPemilik; double saldo; Terdapat tiga atribut yang digunakan untuk menyimpan data rekening, yaitu nomorRekening untuk menyimpan nomor rekening, namaPemilik untuk menyimpan nama pemilik rekening, dan saldo untuk menyimpan jumlah saldo.

public Rekening(String nomor, String nama, double saldoAwal) {
  nomorRekening = nomor;
  namaPemilik = nama;
  saldo = saldoAwal;
 	
  System.out.println(
  "Rekening atas nama " + namaPemilik +
  " berhasil dibuat dengan saldo Rp" + saldo
  );
}
 	
Constructor digunakan untuk memberikan nilai awal pada objek rekening ketika rekening dibuat. Data yang dimasukkan berupa nomor rekening, nama pemilik,

dan saldo awal. Setelah rekening dibuat, program menampilkan informasi bahwa rekening berhasil dibuat.

```
public void setorTunai(double nominal) {
    if (nominal >= 10000) {
        saldo += nominal;

        System.out.println(
            "Setor tunai Rp" + nominal +
            " berhasil. Saldo saat ini: Rp" + saldo
        );
    } else {
        System.out.println(
            "Gagal, nominal setor minimal Rp10.000!"
        );
    }
}
```

Method setorTunai() digunakan untuk melakukan penyetoran uang. Program memeriksa apakah nominal setor minimal Rp10.000. Jika memenuhi syarat, nominal akan ditambahkan ke saldo. Jika kurang dari Rp10.000, transaksi ditolak.

```
public void tarikTunai(double nominal) {
    if (nominal < 10000) {
        System.out.println(
            "Gagal, nominal tarik tunai minimal Rp10.000!"
        );
    } else if (nominal > saldo) {
        System.out.println(
            "Gagal, saldo tidak mencukupi!"
        );
    } else {
        saldo -= nominal;

        System.out.println(
```

```
            "Tarik tunai Rp" + nominal +
            " berhasil. Saldo saat ini: Rp" + saldo
        );
    }
}
```

Method tarikTunai() digunakan untuk melakukan penarikan uang. Penarikan minimal Rp10.000 dan nominal penarikan tidak boleh lebih besar dari saldo yang tersedia. Dengan demikian, saldo rekening tidak akan menjadi minus.

```
public void cekInformasi() {
    System.out.println("\n--- INFO REKENING ---");
    System.out.println("No. Rekening : " + nomorRekening);
    System.out.println("Nama Pemilik : " + namaPemilik);
    System.out.println("Saldo Akhir  : Rp" + saldo);
    System.out.println("----------------------");
}
```

Method cekInformasi() digunakan untuk menampilkan informasi rekening yang sedang aktif, seperti nomor rekening, nama pemilik, dan saldo akhir.

1.2 Program mainRekening

1.2.1 Kode Program Main

```
package pekan1;

import java.util.ArrayList;
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        Scanner input = new Scanner(System.in);
```

```
ArrayList<Rekening> daftarRekening = new ArrayList<>();
Rekening akunAktif = null;

boolean isRunning = true;

System.out.println("=== SISTEM PERBANKAN MINI ===");

while (isRunning) {

    System.out.println("\nMenu Utama:");
    System.out.println("1. Buka Rekening Baru");
    System.out.println("2. Setor Tunai");
    System.out.println("3. Tarik Tunai");
    System.out.println("4. Cek Informasi Rekening");
    System.out.println("5. Ganti Akun");
    System.out.println("0. Keluar");
    System.out.print("Pilih menu: ");

    int pilihan = input.nextInt();
    input.nextLine();

    switch (pilihan) {

        case 1:

            System.out.print("Masukkan No Rekening: ");
            String no = input.nextLine();

            System.out.print("Masukkan Nama Pemilik: ");
            String nama = input.nextLine();

            System.out.print("Masukkan Saldo Awal: ");

            if (input.hasNextDouble()) {

                double saldo = input.nextDouble();

                akunAktif = new Rekening(no, nama, saldo);
                daftarRekening.add(akunAktif);
```

```
            } else {

                System.out.println(
                        "Error: Saldo awal harus berupa angka!");

                input.next();
            }

            break;

        case 2:

            if (akunAktif == null) {

                System.out.println(
                        "Error: Anda belum memiliki rekening!");

            } else {

                System.out.print("Masukkan nominal setor: ");

                if (input.hasNextDouble()) {

                    double setor = input.nextDouble();
                    akunAktif.setorTunai(setor);

                } else {

                    System.out.println(
                            "Error: Nominal harus berupa angka!");

                    input.next();
                }
            }

            break;

        case 3:
```

```
            if (akunAktif == null) {

                System.out.println(
                        "Error: Anda belum memiliki rekening!");

            } else {

                System.out.print("Masukkan nominal tarik: ");

                if (input.hasNextDouble()) {

                    double tarik = input.nextDouble();
                    akunAktif.tarikTunai(tarik);

                } else {

                    System.out.println(
                            "Error: Nominal harus berupa angka!");

                    input.next();
                }
            }

            break;

        case 4:

            if (akunAktif == null) {

                System.out.println(
                        "Error: Anda belum membuka rekening!");

            } else {

                akunAktif.cekInformasi();
            }

            break;

        case 5:
```

```
    System.out.print("Masukkan nomor rekening: ");
    String nomorCari = input.nextLine();

    boolean ditemukan = false;

    for (Rekening rekening : daftarRekening) {

        if (rekening.nomorRekening.equals(nomorCari)) {

            akunAktif = rekening;
            ditemukan = true;

            System.out.println(
                    "Berhasil mengganti akun ke rekening " +
                            akunAktif.namaPemilik);

            break;
        }
    }

    if (!ditemukan) {
        System.out.println("Rekening tidak ditemukan!");
    }

    break;

case 0:

    isRunning = false;

    System.out.println(
            "Sistem ditutup. Terima Kasih!");

    break;

default:

    System.out.println(
            "Pilihan menu tidak tersedia!");
```

```
                break;
            }
        }

        input.close();
    }
}
```

1.2.2 Penjelasan

import java.util.ArrayList; import java.util.Scanner; Scanner digunakan untuk menerima input dari pengguna melalui keyboard. ArrayList digunakan untuk menyimpan beberapa objek rekening sehingga program dapat menangani lebih dari satu rekening.
Scanner input = new Scanner(System.in); ArrayList<Rekening> daftarRekening = new ArrayList<>(); Rekening akunAktif = null; input digunakan untuk membaca input pengguna. daftarRekening digunakan untuk menyimpan seluruh rekening yang telah dibuat. Sedangkan akunAktif digunakan untuk menentukan rekening yang sedang digunakan.

boolean isRunning = true;
 	
while (isRunning) {
 	
while digunakan agar menu utama terus ditampilkan dan dapat digunakan berulang kali sampai pengguna memilih menu keluar.

System.out.println("1. Buka Rekening Baru"); System.out.println("2. Setor Tunai"); System.out.println("3. Tarik Tunai"); System.out.println("4. Cek Informasi Rekening"); System.out.println("5. Ganti Akun");

```
System.out.println("0. Keluar");
```

Menu untuk memberikan beberapa pilihan kepada pengguna, yaitu membuat rekening, melakukan setor tunai, tarik tunai, melihat informasi rekening, mengganti akun, dan keluar dari program.

```
if (!input.hasNextInt()) {
    System.out.println(
        "Error: Pilihan menu harus berupa angka!"
    );

    input.next();
    continue;
}
```

Kode tersebut digunakan untuk memastikan pilihan menu yang dimasukkan berupa angka. Jika pengguna memasukkan huruf atau karakter lain, program menampilkan pesan error dan kembali ke menu utama.

```
Rekening rekeningBaru =
    new Rekening(no, nama, saldo);

daftarRekening.add(rekeningBaru);

akunAktif = rekeningBaru;
```

Program membuat objek rekening baru menggunakan constructor Rekening. Setelah dibuat, objek tersebut ditambahkan ke dalam ArrayList menggunakan add(). Rekening yang baru dibuat kemudian dijadikan sebagai akunAktif. Dengan menggunakan ArrayList, pengguna dapat memiliki **lebih dari satu rekening** tanpa menghapus rekening sebelumnya.

```
double setor = input.nextDouble();

akunAktif.setorTunai(setor);
```

Program menerima nominal setor dari pengguna kemudian memanggil method setorTunai() pada rekening yang sedang aktif.

```
double tarik = input.nextDouble();
```

akunAktif.tarikTunai(tarik);

Program menerima nominal penarikan kemudian memanggil method tarikTunai(). Pemeriksaan minimal Rp10.000 dan saldo yang mencukupi dilakukan di class Rekening.

akunAktif.cekInformasi();

Kode tersebut digunakan untuk menampilkan informasi dari rekening yang sedang aktif.

```
for (Rekening rekening : daftarRekening) {

    if (rekening.nomorRekening.equals(nomorCari)) {

        akunAktif = rekening;
        ditemukan = true;

        break;
    }
}
```

Bagian ini digunakan untuk mencari rekening berdasarkan nomor rekening di dalam ArrayList. Program memeriksa setiap rekening menggunakan for. Jika nomor rekening sesuai, rekening tersebut dijadikan sebagai akunAktif.

```
case 0:

    isRunning = false;

    System.out.println(
        "Sistem ditutup. Terima Kasih!"
    );

    break;
```

Jika kita memilih menu 0, nilai isRunning diubah menjadi false, sehingga perulangan berhenti dan program ditutup.

1.3 Output

```
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 1
Masukkan Nama Pemilik: nafi
Masukkan Saldo Awal: 20000
Rekening atas nama nafi berhasil dibuat dengan saldo Rp20000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu: 2
Masukkan nominal setor: 10000
Setor tunai Rp10000.0 berhasil. Saldo saat ini: Rp30000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu: 3
Masukkan nominal tarik: 5000
Gagal, nominal tarik tunai minimal Rp10.000!
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 2
Masukkan Nama Pemilik: ghifari
Masukkan Saldo Awal: 30000
Rekening atas nama ghifari berhasil dibuat dengan saldo Rp30000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu: 2
Masukkan nominal setor: 10000
Setor tunai Rp10000.0 berhasil. Saldo saat ini: Rp40000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu: 3
Masukkan nominal tarik: 50000
Gagal, saldo tidak mencukupi, saldo Anda : Rp40000.0
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu: 5
Masukkan nomor rekening: 1
Berhasil mengganti akun ke rekening nafi

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu: 5
Masukkan nomor rekening: 2
Berhasil mengganti akun ke rekening ghifari
```

PERTEMUAN 2

2.1 Program Main.java

2.1.1 Kode Program Main.java

```java
package pekan2;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        Rekening akun1 = new Rekening("123", 50000);
        Rekening akun2 = new Rekening("456", 100000);

        Rekening akunAktif = akun1;

        boolean berjalan = true;
        while (berjalan) {
            System.out.println("\n=== MENU UTAMA (Akun: " + akunAktif.getNomorRekening() + ") ===");
            System.out.println("1. Cek Saldo");
            System.out.println("2. Setor Tunai");
            System.out.println("3. Tarik Tunai");
            System.out.println("4. Cek Informasi Rekening");
            System.out.println("5. Ganti Akun");
            System.out.println("6. Cetak Mutasi (Riwayat)");
            System.out.println("7. Informasi Transaksi");
            System.out.println("0. Keluar");
            System.out.print("Pilih menu: ");

            int pilihan = scanner.nextInt();

            switch (pilihan) {
                case 1:
                    System.out.println("Saldo Anda: " + akunAktif.getSaldo());
                    break;
```

```
                case 2:
                    System.out.print("Masukkan nominal setor: ");
                    double nominalSetor = scanner.nextDouble();
                    if (akunAktif.setorTunai(nominalSetor)) {
                        System.out.println("Setor tunai berhasil.");
                    } else {
                        System.out.println("Setor tunai gagal, Maksimal transaksi 500.000");
                    }
                    break;
                case 3:
                    System.out.print("Masukkan nominal tarik: ");
                    double nominalTarik = scanner.nextDouble();
                    if (akunAktif.tarikTunai(nominalTarik)) {
                        System.out.println("Penarikan berhasil.");
                    } else {
                        System.out.println("Penarikan gagal! Saldo tidak mencukupi atau melebihi batas
transaksi 500.000.");
                    }
                    break;
                case 4:
                    System.out.println("\n--- INFORMASI REKENING ---");
                    System.out.println("Nomor Rekening : " + akunAktif.getNomorRekening());
                    System.out.println("Saldo          : " + akunAktif.getSaldo());
                    break;
                case 5:
                    if (akunAktif == akun1) {
                        akunAktif = akun2;
                    } else {
                        akunAktif = akun1;
                    }
                    System.out.println("Berhasil berganti akun! Akun aktif saat ini: " +
akunAktif.getNomorRekening());
                    break;
                case 6:
                    akunAktif.cetakMutasi();
                    break;
                case 7:
                    akunAktif.cetakInformasi();
                    break;
                case 0:
                    berjalan = false;
                    System.out.println("Terima kasih telah menggunakan layanan kami.");
                    break;
                default:
                    System.out.println("Pilihan tidak valid.");
            }
        }
        scanner.close();
    }
}
```

2.1.2 Penjelasan

a. Membuat dua objek dari class yang sama

Rekening akun1 = new Rekening("123", 50000);
Rekening akun2 = new Rekening("456", 100000);

Baris ini menginstansiasi dua objek Rekening yang berbeda dari class yang sama. Masing-masing objek memiliki nomor rekening dan saldo awal sendiri, membuktikan bahwa satu class dapat menghasilkan banyak objek dengan data (state) yang terpisah.

b. Variabel penunjuk akun yang sedang aktif

```
Rekening akunAktif = akun1;
```

akunAktif adalah variabel referensi yang menunjuk ke salah satu objek Rekening. Karena hanya berisi referensi (bukan salinan data), mengganti nilai akunAktif cukup membuatnya menunjuk ke objek lain — inilah dasar dari fitur "Ganti Akun".

c. Perulangan menu utama

```
while (berjalan) {
    ... tampilkan menu ...
}
```

Perulangan while digunakan agar menu terus ditampilkan berulang kali. Perulangan ini baru berhenti ketika variabel boolean berjalan diubah menjadi false, yaitu saat pengguna memilih menu 0 (Keluar).

d. Percabangan berdasarkan pilihan menu

```
switch (pilihan) {
    case 1: ... break;
    case 2: ... break;
    ...
}
```

Struktur switch-case memetakan angka yang diinput pengguna ke aksi yang sesuai. Setiap case memanggil method milik akunAktif (misalnya akunAktif.getSaldo()), sehingga Main tidak perlu tahu detail perhitungan — cukup memanggil method yang sudah disediakan class Rekening.

e. Validasi hasil transaksi

```
if (akunAktif.setorTunai(nominalSetor)) {
    System.out.println("Setor tunai berhasil.");
} else {
    System.out.println("Setor tunai gagal, Maksimal transaksi 500.000");
}
```

Method setorTunai() mengembalikan nilai boolean (true/false). Main hanya perlu memeriksa hasilnya untuk menampilkan pesan berhasil atau gagal, sedangkan seluruh logika validasi nominal disimpan di dalam class Rekening.

f. Mekanisme ganti akun

```
if (akunAktif == akun1) {
    akunAktif = akun2;
} else {
    akunAktif = akun1;
}
```

Operator == di sini membandingkan referensi objek (apakah akunAktif sedang menunjuk ke objek yang persis sama dengan akun1), bukan membandingkan isi datanya. Jika ya, akunAktif dialihkan ke akun2, dan sebaliknya.

2.2 Program Rekening.java

2.2.1 Kode Program Rekening.java

```java
package pekan2;

import java.util.ArrayList;

public class Rekening {
    private String nomorRekening;
    private double saldoAwal;
    private ArrayList<Transaksi> riwayatTransaksi;

    public Rekening(String nomorRekening, double saldoAwal) {
        this.nomorRekening = nomorRekening;
        this.saldoAwal = saldoAwal;
        this.riwayatTransaksi = new ArrayList<>();
    }

    public boolean tarikTunai(double nominal) {
        if (nominal > 0 && nominal <= 500000 && nominal <= getSaldo()) {
            String kodeTransaksi = "TRX-T-" + (riwayatTransaksi.size() + 1);
            Transaksi trx = new Transaksi(kodeTransaksi, "Debit", nominal);
            riwayatTransaksi.add(trx);
            return true;
        } else {
            return false;
        }
    }
```

```java
    public boolean setorTunai(double nominal) {
        if (nominal > 0 && nominal <= 500000) {
            String kodeTransaksi = "TRX-S-" + (riwayatTransaksi.size() + 1);
            Transaksi trx = new Transaksi(kodeTransaksi, "Kredit", nominal);
            riwayatTransaksi.add(trx);
            return true;
        } else {
            return false;
        }
    }

    public void cetakMutasi() {
        if (riwayatTransaksi.isEmpty()) {
            System.out.println("Belum ada transaksi pada rekening ini");
        } else {
            for (Transaksi trx : riwayatTransaksi) {
                trx.cetakDetail();
            }
        }
    }

    public void cetakInformasi() {
        double totalSetor = 0;
        double totalTarik = 0;

        for (Transaksi trx : riwayatTransaksi) {
            if (trx.getJenisTransaksi().equalsIgnoreCase("Kredit")) {
                totalSetor += trx.getNominal();
            } else if (trx.getJenisTransaksi().equalsIgnoreCase("Debit")) {
                totalTarik += trx.getNominal();
            }
        }

        double akumulasi = totalSetor - totalTarik;
        double saldoAkhir = saldoAwal + akumulasi;

        System.out.println("\n--- INFO TRANSAKSI ---");
        System.out.println("Total Setor : Rp " + totalSetor);
        System.out.println("Total Tarik : Rp " + totalTarik);
        System.out.println("Akumulasi   : Rp " + akumulasi);
        System.out.println("Saldo : Rp " + saldoAkhir);
    }
```



```
public double getSaldo() {
        double akumulasi = 0;
        for (Transaksi trx : riwayatTransaksi) {
            if (trx.getJenisTransaksi().equalsIgnoreCase("Kredit")) {
                akumulasi += trx.getNominal();
            } else if (trx.getJenisTransaksi().equalsIgnoreCase("Debit")) {
                akumulasi -= trx.getNominal();
            }
        }
        return saldoAwal + akumulasi;
    }

    public String getNomorRekening() {
        return nomorRekening;
    }
}
```

2.2.2 Penjelasan

a. Atribut private (enkapsulasi)

private String nomorRekening;
private double saldoAwal;
private ArrayList<Transaksi> riwayatTransaksi;

Ketiga atribut dideklarasikan private sehingga tidak dapat diakses langsung dari class lain seperti Main. Data hanya bisa dibaca atau diubah melalui method publik yang disediakan (getSaldo(), setorTunai(), dan seterusnya) — inilah penerapan enkapsulasi.

b. Validasi dan pencatatan penarikan

```
if (nominal > 0 && nominal <= 500000 && nominal <= getSaldo()) {
    String kodeTransaksi = "TRX-T-" + (riwayatTransaksi.size() + 1);
    Transaksi trx = new Transaksi(kodeTransaksi, "Debit", nominal);
    riwayatTransaksi.add(trx);
    return true;
} else {
    return false;
}
```

Method tarikTunai() hanya meloloskan transaksi jika nominal positif, tidak melebihi Rp 500.000, dan tidak melebihi saldo saat ini. Jika lolos, dibuat objek Transaksi baru berjenis "Debit" dan disimpan ke riwayatTransaksi; saldo tidak diubah langsung, melainkan akan dihitung ulang dari riwayat ini.

c. Validasi dan pencatatan setoran

```
if (nominal > 0 && nominal <= 500000) {
   String kodeTransaksi = "TRX-S-" + (riwayatTransaksi.size() + 1);
   Transaksi trx = new Transaksi(kodeTransaksi, "Kredit", nominal);
   riwayatTransaksi.add(trx);
   return true;
}
```

Method setorTunai() mirip dengan tarikTunai(), namun tidak perlu mengecek kecukupan saldo — hanya nominal positif dan tidak melebihi batas Rp 500.000. Transaksi yang tercatat berjenis "Kredit".

d. Perhitungan saldo secara akumulatif

```
double akumulasi = 0;
for (Transaksi trx : riwayatTransaksi) {
   if (trx.getJenisTransaksi().equalsIgnoreCase("Kredit")) {
      akumulasi += trx.getNominal();
   } else if (trx.getJenisTransaksi().equalsIgnoreCase("Debit")) {
      akumulasi -= trx.getNominal();
   }
}
return saldoAwal + akumulasi;
```

Program ini tidak menyimpan saldo sebagai satu variabel yang diubah-ubah, melainkan menghitungnya ulang setiap kali getSaldo() dipanggil, dengan menelusuri seluruh riwayatTransaksi: menjumlahkan semua Kredit dan mengurangkan semua Debit, lalu ditambahkan ke saldoAwal. Pendekatan ini membuat saldo selalu konsisten dengan histori transaksi yang tercatat.

e. Mencetak riwayat transaksi

```
for (Transaksi trx : riwayatTransaksi) {
   trx.cetakDetail();
}
```

cetakMutasi() menelusuri seluruh objek Transaksi dalam riwayatTransaksi dan memanggil method cetakDetail() milik masing-masing objek untuk menampilkan detailnya satu per satu.

2.3 Program Transaksi.java

2.3.1 Kode Program Transaksi.java

```java
package pekan2;

public class Transaksi {
    String kodeTransaksi;
    String jenisTransaksi;
    double nominal;

    public Transaksi(String kodeTransaksi, String jenisTransaksi, double nominal) {
        this.kodeTransaksi = kodeTransaksi;
        this.jenisTransaksi = jenisTransaksi;
        this.nominal = nominal;
    }

    public String getKodeTransaksi() {
        return kodeTransaksi;
    }

    public String getJenisTransaksi() {
        return jenisTransaksi;
    }

    public double getNominal() {
        return nominal;
    }

    public void cetakDetail() {
        System.out.println(kodeTransaksi + " | " + jenisTransaksi + " | Rp " +
nominal);
    }
}
```

2.3.2 Penjelasan

a. Atribut penyusun satu transaksi
String kodeTransaksi;
String jenisTransaksi;
double nominal;
Satu objek Transaksi hanya merepresentasikan satu baris transaksi: kode unik, jenis ("Kredit" atau "Debit"), dan nominalnya. Karena disimpan di dalam ArrayList milik Rekening, satu objek Rekening dapat memiliki banyak objek Transaksi (hubungan one-to-many).

b. Mencetak detail satu transaksi

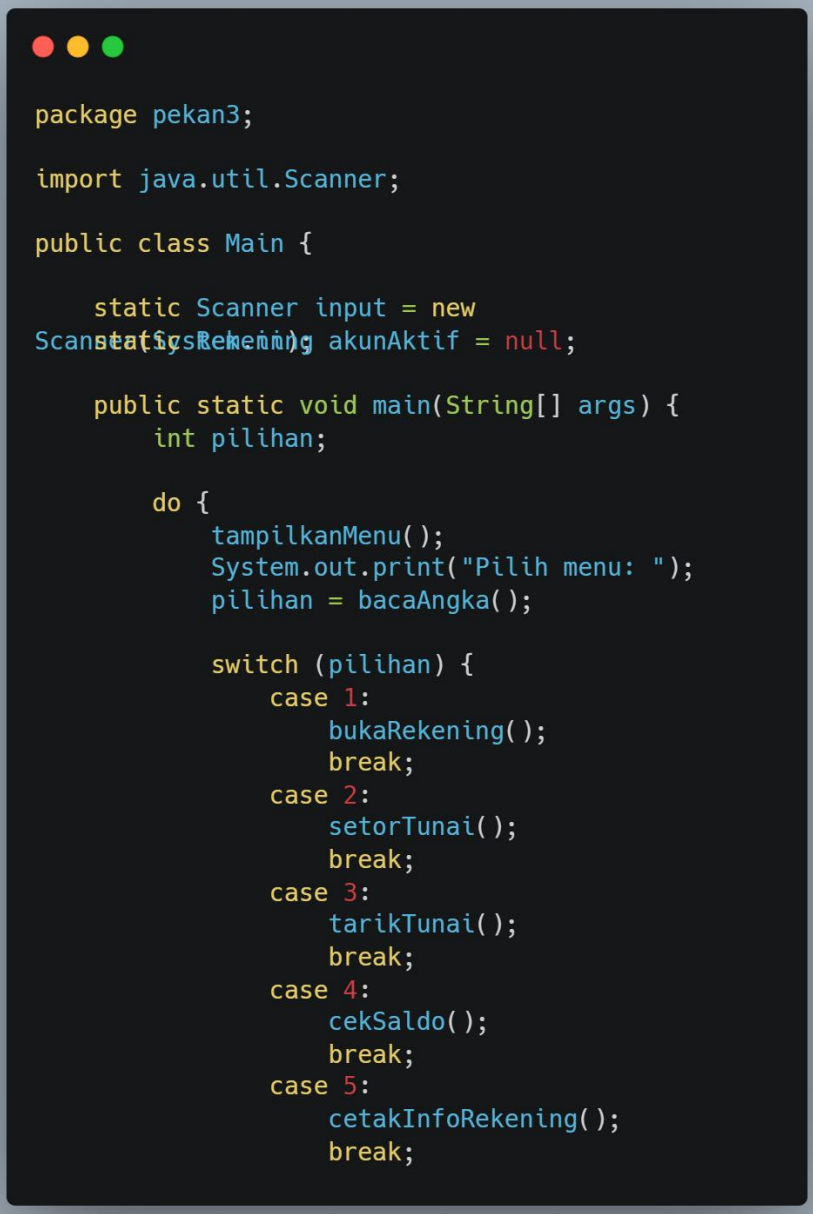
```
public void cetakDetail() {
    System.out.println(kodeTransaksi + " | " + jenisTransaksi + " | Rp " +
nominal);
}
```

Method ini dipanggil berulang kali oleh Rekening.cetakMutasi() untuk mencetak setiap transaksi dalam format kode | jenis | nominal, sehingga riwayat transaksi mudah dibaca.

PERTEMUAN 3

3.1 Program Main.java

3.1.1 Kode Program Main.java



```java
package pekan3;

import java.util.Scanner;

public class Main {

    static Scanner input = new
Scanner(System.in);
    static Rekening akunAktif = null;

    public static void main(String[] args) {
        int pilihan;

        do {
            tampilkanMenu();
            System.out.print("Pilih menu: ");
            pilihan = bacaAngka();

            switch (pilihan) {
                case 1:
                    bukaRekening();
                    break;
                case 2:
                    setorTunai();
                    break;
                case 3:
                    tarikTunai();
                    break;
                case 4:
                    cekSaldo();
                    break;
                case 5:
                    cetakInfoRekening();
                    break;
```



```
                case 6:
                    cetakMutasi();
                    break;
                case 7:
                    System.out.println("Terima kasih telah menggunakan layanan kami. Sampai
jumpa!");
                    break;
                default:
                    System.out.println("Pilihan tidak valid. Silakan coba lagi.");
            }

            System.out.println();

        } while (pilihan != 7);

        input.close();
    }

    static void tampilkanMenu() {
        System.out.println("===== MENU BANK =====");
        System.out.println("1. Buka Rekening");
        System.out.println("2. Setor Tunai");
        System.out.println("3. Tarik Tunai");
        System.out.println("4. Cek Saldo");
        System.out.println("5. Info Rekening");
        System.out.println("6. Cetak Mutasi (Riwayat Transaksi)");
        System.out.println("7. Keluar");
    }

    static int bacaAngka() {
        while (!input.hasNextInt()) {
            System.out.print("Masukkan angka yang valid: ");
            input.next();
        }
        int angka = input.nextInt();
        input.nextLine(); // buang sisa newline
        return angka;
    }
```

```java
    // 1. Buka Rekening — minta PIN sebelum instansiasi objek baru
    static void bukaRekening() {
        System.out.print("Masukkan nomor rekening: ");
        String nomor = input.nextLine();

        System.out.print("Masukkan nama pemilik: ");
        String nama = input.nextLine();

        System.out.print("Masukkan saldo awal: ");
        double saldoAwal = Double.parseDouble(bacaAngkaDesimal());

        String pin;
        while (true) {
            System.out.print("Masukkan PIN (6 digit angka): ");
            pin = input.nextLine();
            if (pin.length() == 6 && pin.chars().allMatch(Character::isDigit)) {
                break;
            }
            System.out.println("PIN tidak valid! PIN harus terdiri dari 6 digit angka.");
        }

        akunAktif = new Rekening(nomor, nama, saldoAwal, pin);
    }

    static String bacaAngkaDesimal() {
        while (!input.hasNextDouble()) {
            System.out.print("Masukkan angka yang valid: ");
            input.next();
        }
        String nilai = String.valueOf(input.nextDouble());
        input.nextLine(); // buang sisa newline
        return nilai;
    }

    // 2. Setor Tunai
    static void setorTunai() {
        if (!adaAkunAktif()) return;

        System.out.print("Masukkan nominal setor: ");
        double nominal = Double.parseDouble(bacaAngkaDesimal());
        akunAktif.setor(nominal);
    }

    // 3. Tarik Tunai — wajib otentikasi PIN dulu
    static void tarikTunai() {
        if (!adaAkunAktif()) return;

        System.out.print("Masukkan PIN Anda: ");
        String pinYangDiinput = input.nextLine();

        if (akunAktif.otentikasi(pinYangDiinput)) {
            System.out.print("Masukkan nominal tarik: ");
            double nominal = Double.parseDouble(bacaAngkaDesimal());
            akunAktif.tarik(nominal);
        } else {
            System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");
        }
    }

    // 4. Cek Saldo
    static void cekSaldo() {
        if (!adaAkunAktif()) return;
        System.out.println("Saldo saat ini: Rp" + akunAktif.getSaldo());
    }

    // 5. Info Rekening
    static void cetakInfoRekening() {
        if (!adaAkunAktif()) return;
        akunAktif.cetakInfo();
    }

    // 6. Cetak Mutasi — wajib otentikasi PIN dulu
    static void cetakMutasi() {
        if (!adaAkunAktif()) return;

        System.out.print("Masukkan PIN Anda: ");
        String pinYangDiinput = input.nextLine();

        if (akunAktif.otentikasi(pinYangDiinput)) {
            System.out.println("--- Riwayat Transaksi ---");
            if (akunAktif.getRiwayatTransaksi().isEmpty()) {
                System.out.println("Belum ada transaksi.");
            } else {
                for (Transaksi t : akunAktif.getRiwayatTransaksi()) {
                    t.cetakDetail();
                }
            }
        } else {
            System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");
        }
    }

    static boolean adaAkunAktif() {
        if (akunAktif == null) {
            System.out.println("Belum ada rekening aktif. Silakan buka rekening terlebih dahulu (Menu
1).");
            return false;
        }
        return true;
    }
}
```

3.1.2 Penjelasan

1. Class Transaksi berada pada package pekan3 dan berperan sebagai class sederhana untuk merepresentasikan satu objek transaksi (data holder), dengan tiga atribut: idTransaksi, jenis, dan nominal. Ketiga atribut ini menggunakan modifier akses default (package-private), yaitu hanya dapat diakses dari class lain yang berada pada package yang sama.

2. Constructor Transaksi(String id, String jenis, double nominal) menginisialisasi ketiga atribut menggunakan kata kunci this untuk membedakan atribut class dengan parameter yang memiliki nama sama.

3. Disediakan tiga getter method publik (getIdTransaksi(), getJenis(), getNominal()) yang memungkinkan class lain (seperti Main) membaca nilai atribut objek Transaksi tanpa mengubahnya secara langsung.

4. Method cetakDetail() mencetak representasi satu baris dari objek transaksi, yang dipanggil dari Main.cetakMutasi() untuk menampilkan setiap item riwayat transaksi.

5. Class ini tidak memiliki atribut private maupun banyak validasi karena perannya murni sebagai objek data yang dibuat dan disimpan oleh class Rekening setiap kali terjadi transaksi setor atau tarik.

3.2 Program Rekening.java

3.2.1 Kode Program Rekening.java

```java
package pekan3;

import java.util.ArrayList;

public class Rekening {
    private String nomorRekening;
    private String namaPemilik;
    private double saldo;
    private String pin;

    private ArrayList<Transaksi> riwayatTransaksi;

    public Rekening(String nomor, String nama, double saldoAwal, String pinAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;

        if (pinAwal.length() == 6) {
            this.pin = pinAwal;
        } else {
            System.out.println("Peringatan: PIN harus 6 digit! Menggunakan PIN default
123456");   this.pin = "123456";
        }
        this.riwayatTransaksi = new ArrayList<>();
        System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat.");
    }

    public String getNomorRekening() {
        return nomorRekening;
    }
```

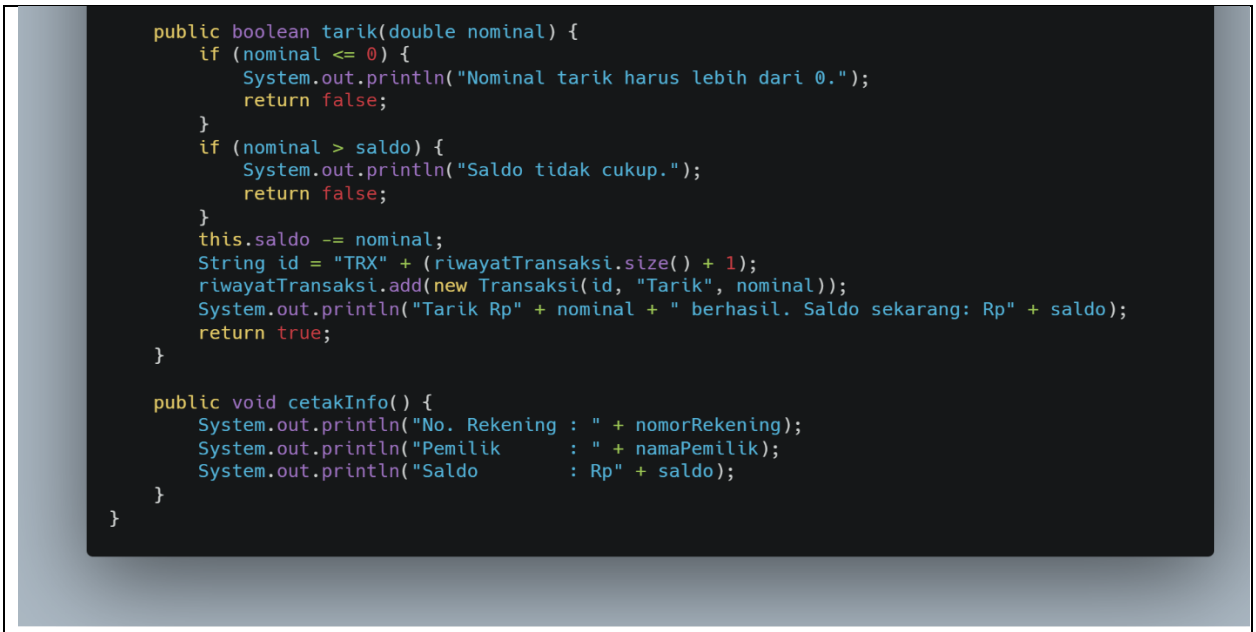
```java
    public String getNamaPemilik() {
        return namaPemilik;
    }

    public double getSaldo() {
        return saldo;
    }

    public ArrayList<Transaksi> getRiwayatTransaksi() {
        return riwayatTransaksi;
    }

    public boolean otentikasi(String inputPin) {
        return this.pin.equals(inputPin);
    }

    public boolean setor(double nominal) {
        if (nominal <= 0) {
            System.out.println("Nominal setor harus lebih dari 0.");
            return false;
        }
        this.saldo += nominal;
        String id = "TRX" + (riwayatTransaksi.size() + 1);
        riwayatTransaksi.add(new Transaksi(id, "Setor", nominal));
        System.out.println("Setor Rp" + nominal + " berhasil. Saldo sekarang: Rp" + saldo);
        return true;
    }
```



```
    public boolean tarik(double nominal) {
        if (nominal <= 0) {
            System.out.println("Nominal tarik harus lebih dari 0.");
            return false;
        }
        if (nominal > saldo) {
            System.out.println("Saldo tidak cukup.");
            return false;
        }
        this.saldo -= nominal;
        String id = "TRX" + (riwayatTransaksi.size() + 1);
        riwayatTransaksi.add(new Transaksi(id, "Tarik", nominal));
        System.out.println("Tarik Rp" + nominal + " berhasil. Saldo sekarang: Rp" + saldo);
        return true;
    }

    public void cetakInfo() {
        System.out.println("No. Rekening : " + nomorRekening);
        System.out.println("Pemilik      : " + namaPemilik);
        System.out.println("Saldo        : Rp" + saldo);
    }
}
```

3.2.2 Penjelasan

1. Class Rekening mendeklarasikan seluruh atributnya (nomorRekening, namaPemilik, saldo, pin, riwayatTransaksi) sebagai private, sehingga menerapkan prinsip enkapsulasi secara penuh — atribut ini hanya dapat diakses dari luar class melalui getter method atau method publik lain yang telah disediakan.

2. Atribut riwayatTransaksi bertipe ArrayList<Transaksi> menunjukkan relasi komposisi: satu objek Rekening memiliki dan mengelola banyak objek Transaksi sekaligus.

3. Constructor Rekening(...) menginisialisasi nomor rekening, nama pemilik, dan saldo awal, kemudian memvalidasi PIN yang dimasukkan: jika panjangnya tepat 6 karakter maka digunakan sebagai PIN, jika tidak maka program mencetak peringatan dan menetapkan PIN default "123456". Di akhir constructor, riwayatTransaksi diinisialisasi sebagai ArrayList kosong, dan pesan konfirmasi pembuatan rekening dicetak.

4. Method otentikasi(String inputPin) membandingkan PIN tersimpan dengan PIN yang diinput menggunakan equals(), dan mengembalikan nilai boolean

sebagai dasar validasi sebelum operasi sensitif (tarik tunai dan cetak mutasi) dapat dilakukan.

5. Method setor(double nominal) dan tarik(double nominal) masing-masing memvalidasi nominal transaksi (harus > 0, dan untuk tarik tunai saldo harus mencukupi) sebelum memperbarui saldo, membuat objek Transaksi baru dengan ID otomatis ("TRX" + urutan), menambahkannya ke riwayatTransaksi, lalu mencetak konfirmasi. Kedua method mengembalikan nilai boolean untuk menandakan keberhasilan transaksi.

6. Method cetakInfo() menampilkan ringkasan data rekening (nomor, pemilik, saldo) yang dipanggil dari Main.cetakInfoRekening().

3.3 Program Transaksi.java

3.3.1 Kode Program Transaksi.java

```java
package pekan3;

public class Transaksi {
    String idTransaksi;
    String jenis;
    double nominal;

    public Transaksi(String id, String jenis, double nominal) {
        this.idTransaksi = id;
        this.jenis = jenis;
        this.nominal = nominal;
    }

    public String getIdTransaksi() { return idTransaksi; }
    public String getJenis() { return jenis; }
    public double getNominal() { return nominal; }

    public void cetakDetail() {
        System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" +
nominal);
}
```

3.3.2 Penjelasan

1. Atribut static: input (Scanner) dan akunAktif (objek Rekening yang sedang login, null di awal).

2. main(): loop do-while yang memanggil tampilkanMenu() → bacaAngka() untuk baca pilihan → switch-case menjalankan method sesuai nomor menu, berulang sampai pilihan = 7 (Keluar).

3. bacaAngka()/bacaAngkaDesimal(): validasi input agar hanya menerima angka, memakai loop while (!input.hasNextInt()) sehingga input salah tidak membuat program *crash*.

4. bukaRekening() (Menu 1): mengumpulkan data, validasi PIN dalam while(true) sampai valid (6 digit angka via pin.chars().allMatch(Character::isDigit)), baru kemudian new Rekening(...) dipanggil — di sinilah constructor Rekening dieksekusi.

5. setorTunai() / tarikTunai() / cekSaldo() / cetakInfoRekening() / cetakMutasi() (Menu 2–6): masing-masing cek dulu adaAkunAktif() (mencegah NullPointerException bila rekening belum dibuat), lalu memanggil method terkait pada akunAktif. Khusus tarikTunai() dan cetakMutasi(), wajib lolos akunAktif.otentikasi(pin) dulu. Di cetakMutasi(), program melakukan for (Transaksi t : akunAktif.getRiwayatTransaksi()) untuk mencetak setiap riwayat transaksi satu per satu.

6. adaAkunAktif(): helper yang mengecek akunAktif == null, mencegah error saat pengguna memilih menu 2–6 sebelum membuka rekening.

3.4 Output

Ketika membuka rekening dia meminta PIN 6 Digit

```
===== MENU BANK =====
1. Buka Rekening
2. Setor Tunai
3. Tarik Tunai
4. Cek Saldo
5. Info Rekening
6. Cetak Mutasi (Riwayat Transaksi)
7. Keluar
Pilih menu: 1
Masukkan nomor rekening: 123
Masukkan nama pemilik: nafi
Masukkan saldo awal: 10000
Masukkan PIN (6 digit angka): 111111
Rekening atas nama nafi berhasil dibuat.
```

Ketika Tarik tunai, dia akan meminta PIN sebelum memasukkan nominal tarik

```
===== MENU BANK =====
1. Buka Rekening
2. Setor Tunai
3. Tarik Tunai
4. Cek Saldo
5. Info Rekening
6. Cetak Mutasi (Riwayat Transaksi)
7. Keluar
Pilih menu: 3
Masukkan PIN Anda: 111111
Masukkan nominal tarik: 5000
Tarik Rp5000.0 berhasil. Saldo sekarang: Rp5000.0
```

Ketika mencetak mutasi (Riwayat Transaksi) dia akan meminta pin terlebih dahulu

```
===== MENU BANK =====
1. Buka Rekening
2. Setor Tunai
3. Tarik Tunai
4. Cek Saldo
5. Info Rekening
6. Cetak Mutasi (Riwayat Transaksi)
7. Keluar
Pilih menu: 6
Masukkan PIN Anda: 111111
--- Riwayat Transaksi ---
ID: TRX1 | Jenis: Tarik | Nominal: Rp5000.0
```

Ketika salah memasukkan PIN, maka ada tulisan “PIN yang Anda masukkan salah!”

```
Pilih menu: 3
Masukkan PIN Anda: 234567
Akses Ditolak: PIN yang Anda masukkan salah!
```

PERTEMUAN 4

4.1 Program Main.java

4.2.1 Kode Program Main.java

```java
package pekan4;

import java.util.ArrayList;
import java.util.Scanner;

public class Main {
```

```
    public static void main(String[] args) {

        Scanner input = new Scanner(System.in);

        ArrayList<Rekening> daftarRekening = new ArrayList<>();
        Rekening akunAktif = null;

        boolean isRunning = true;

        System.out.println("=== SISTEM PERBANKAN MINI ===");

        while (isRunning) {

            System.out.println("\nMenu Utama:");
            System.out.println("1. Buka Rekening Baru");
            System.out.println("2. Setor Tunai");
            System.out.println("3. Tarik Tunai");
            System.out.println("4. Cek Informasi Rekening");
            System.out.println("5. Ganti Akun");
            System.out.println("7. Simulasi Akhir Bulan (Khusus Tabungan)");
            System.out.println("8. Simulasi Akhir Tahun (Khusus Tabungan)"); //
Menu Baru
            System.out.println("0. Keluar");
            System.out.print("Pilih menu: ");

            int pilihan = input.nextInt();
            input.nextLine();

            switch (pilihan) {

                case 1:

                    System.out.print("Masukkan No Rekening: ");
                    String no = input.nextLine();

                    System.out.print("Masukkan Nama Pemilik: ");
                    String nama = input.nextLine();

                    System.out.print("Masukkan Saldo Awal: ");

                    if (!input.hasNextDouble()) {
                        System.out.println("Error: Saldo awal harus berupa
angka!");
                        input.next();
                        break;
```

```
                    }

                    double saldoAwal = input.nextDouble();
                    input.nextLine();

                    System.out.print("Masukkan PIN: ");
                    String pinAwal = input.nextLine();

                    System.out.println("Pilih Produk: 1. Tabungan Umum | 2.
Giro Bisnis");
                    System.out.print("Pilihan: ");
                    int produk = input.nextInt();
                    input.nextLine();

                    Rekening rekeningBaru = null;

                    if (produk == 1) {
                        System.out.print("Masukkan Suku Bunga (%): ");
                        double sukuBunga = input.nextDouble();
                        input.nextLine();
                        rekeningBaru = new RekeningTabungan(no, nama,
saldoAwal, pinAwal, sukuBunga);

                    } else if (produk == 2) {
                        System.out.print("Masukkan Batas Overdraft: ");
                        double batasOverdraft = input.nextDouble();
                        input.nextLine();
                        rekeningBaru = new RekeningGiro(no, nama, saldoAwal,
pinAwal, batasOverdraft);

                    } else {
                        System.out.println("Produk tidak valid, rekening tidak
dibuat.");
                        break;
                    }

                    // Upcasting: RekeningTabungan / RekeningGiro otomatis
dikenali sebagai Rekening
                    daftarRekening.add(rekeningBaru);
                    akunAktif = rekeningBaru;
                    break;

                case 2:
                    if (akunAktif == null) {
```

```
                        System.out.println("Error: Anda belum memiliki
rekening!");
                    } else {
                        System.out.print("Masukkan nominal setor: ");
                        if (input.hasNextDouble()) {
                            double setor = input.nextDouble();
                            akunAktif.setor(setor);
                        } else {
                            System.out.println("Error: Nominal harus berupa
angka!");
                            input.next();
                        }
                    }
                    break;

                case 3:
                    if (akunAktif == null) {
                        System.out.println("Error: Anda belum memiliki
rekening!");
                    } else {
                        System.out.print("Masukkan nominal tarik: ");
                        if (input.hasNextDouble()) {
                            double tarik = input.nextDouble();
                            akunAktif.tarik(tarik);
                        } else {
                            System.out.println("Error: Nominal harus berupa
angka!");
                            input.next();
                        }
                    }
                    break;

                case 4:
                    if (akunAktif == null) {
                        System.out.println("Error: Anda belum membuka
rekening!");
                    } else {
                        akunAktif.cetakInfo();
                    }
                    break;

                case 5:
                    System.out.print("Masukkan nomor rekening: ");
                    String nomorCari = input.nextLine();
                    boolean ditemukan = false;
```

```
                    for (Rekening rekening : daftarRekening) {
                        if (rekening.getNomorRekening().equals(nomorCari)) {
                            akunAktif = rekening;
                            ditemukan = true;
                            System.out.println("Berhasil mengganti akun ke
rekening " + akunAktif.getNamaPemilik());
                            break;
                        }
                    }

                    if (!ditemukan) {
                        System.out.println("Rekening tidak ditemukan!");
                    }
                    break;

                case 7:
                    if (akunAktif == null) {
                        System.out.println("Error: Anda belum memiliki rekening
aktif!");
                    } else if (akunAktif instanceof RekeningTabungan) {
                        // Downcasting: aman karena sudah dicek dengan
instanceof
                        RekeningTabungan tab = (RekeningTabungan) akunAktif;
                        tab.tambahBungaAkhirBulan();
                    } else {
                        System.out.println("Gagal: Fitur bunga akhir bulan
hanya berlaku untuk Rekening Tabungan.");
                    }
                    break;

                case 8:
                    if (akunAktif == null) {
                        System.out.println("Error: Anda belum memiliki rekening
aktif!");
                    } else if (akunAktif instanceof RekeningTabungan) {
                        RekeningTabungan tab = (RekeningTabungan) akunAktif;
                        tab.tambahBungaAkhirTahun();
                    } else {
                        System.out.println("Gagal: Fitur bunga akhir tahun
hanya berlaku untuk Rekening Tabungan.");
                    }
                    break;

                case 0:
```

```
                    isRunning = false;
                    System.out.println("Sistem ditutup. Terima Kasih!");
                    break;

                default:
                    System.out.println("Pilihan menu tidak tersedia!");
                    break;
            }
        }
        input.close();
    }
}
```

2.2.2 Penjelasan

- Menampilkan menu interaktif utama menggunakan *looping* while dan struktur kontrol switch-case.
- Mengelola sekumpulan akun rekening menggunakan koleksi ArrayList<Rekening>.

- **Upcasting:** Saat pembuatan rekening baru (opsi 1), objek RekeningTabungan atau RekeningGiro dimasukkan ke dalam daftar ber tipe Rekening.
- **Downcasting & Instanceof:** Pada simulasi akhir bulan (opsi 7) dan akhir tahun (opsi 8), program memeriksa tipe objek menggunakan keyword instanceof untuk memastikan akun aktif adalah RekeningTabungan sebelum melakukan *casting* ke tipe spesifik tersebut.

4.2 Program Rekening.java

4.2.1 Kode Program Rekening.java

```
package pekan4;

import java.util.ArrayList;

public class Rekening {
    private String nomorRekening;
    private String namaPemilik;
    private String pin;
```

```
    protected double saldo;
    protected ArrayList<Transaksi> riwayatTransaksi;

    public Rekening(String nomor, String nama, double saldoAwal, String
pinAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;

        if (pinAwal.length() == 6) {
            this.pin = pinAwal;
        } else {
            System.out.println("Peringatan: PIN harus 6 digit! Menggunakan PIN
default 123456");
            this.pin = "123456";
        }
        this.riwayatTransaksi = new ArrayList<>();
        System.out.println("Rekening atas nama " + namaPemilik + " berhasil
dibuat.");
    }

    public String getNomorRekening() {
        return nomorRekening;
    }

    public String getNamaPemilik() {
        return namaPemilik;
    }

    public double getSaldo() {
        return saldo;
    }

    public ArrayList<Transaksi> getRiwayatTransaksi() {
        return riwayatTransaksi;
    }

    public boolean otentikasi(String inputPin) {
        return this.pin.equals(inputPin);
    }

    public boolean setor(double nominal) {
        if (nominal <= 0) {
            System.out.println("Nominal setor harus lebih dari 0.");
            return false;
```

```
        }
        this.saldo += nominal;
        String id = "TRX" + (riwayatTransaksi.size() + 1);
        riwayatTransaksi.add(new Transaksi(id, "Setor", nominal));
        System.out.println("Setor Rp" + nominal + " berhasil. Saldo sekarang:
Rp" + saldo);
        return true;
    }

    public boolean tarik(double nominal) {
        if (nominal <= 0) {
            System.out.println("Nominal tarik harus lebih dari 0.");
            return false;
        }
        if (nominal > saldo) {
            System.out.println("Saldo tidak cukup.");
            return false;
        }
        this.saldo -= nominal;
        String id = "TRX" + (riwayatTransaksi.size() + 1);
        riwayatTransaksi.add(new Transaksi(id, "Tarik", nominal));
        System.out.println("Tarik Rp" + nominal + " berhasil. Saldo sekarang:
Rp" + saldo);
        return true;
    }

    public void cetakInfo() {
        System.out.println("No. Rekening : " + nomorRekening);
        System.out.println("Pemilik      : " + namaPemilik);
        System.out.println("Saldo        : Rp" + saldo);
    }
}
```

2.3.2 Penjelasan

- nomorRekening & namaPemilik: Menyimpan identitas pemilik rekening (akses private).
- pin: Menyimpan PIN 6 digit untuk keamanan otentikasi. Jika PIN tidak 6 digit, otomatis diset ke nilai *default* 123456.
- saldo: Menyimpan jumlah saldo (akses protected agar dapat diakses langsung oleh kelas turunannya).
- riwayatTransaksi: Daftar berjenis ArrayList<Transaksi> untuk mencatat seluruh riwayat aktivitas transaksi.

 setor(double nominal): Menambahkan saldo dan mencatat transaksi ke riwayat.
 tarik(double nominal): Mengurangi saldo jika nominal valid dan saldo mencukupi.
 otentikasi(String inputPin): Memvalidasi kesesuaian PIN.
 cetakInfo(): Menampilkan informasi nomor rekening, nama pemilik, dan saldo saat ini.

4.3 Program Transaksi.java

4.3.1 Kode Program Transaksi.java

```
package pekan4;

public class Transaksi {

    private String idTransaksi;
    private String jenisTransaksi;
    private double nominal;

    public Transaksi(String idTransaksi, String jenisTransaksi, double nominal)
{
        this.idTransaksi = idTransaksi;
        this.jenisTransaksi = jenisTransaksi;
        this.nominal = nominal;
    }

    @Override
    public String toString() {
        return "[" + idTransaksi + "] " + jenisTransaksi + " sebesar Rp" +
nominal;
    }
}
```

3.4.1 Penjelasan

- idTransaksi: Kode unik transaksi.
- jenisTransaksi: Deskripsi tipe transaksi (misal: "Setor", "Tarik Giro", "Bunga Bulanan").
- nominal: Jumlah uang dalam transaksi tersebut.

- toString() (*Overriding*): Mengubah format tampilan objek Transaksi menjadi teks berformat rapi, contoh: [TRX1] Setor sebesar Rp500000.0.

4.4 Program RekeningGiro

4.4.1 Kode Program RekeningGiro.java

```
package pekan4;

public class RekeningGiro extends Rekening {
    private double batasOverdraft;

    public RekeningGiro(String nomor, String nama, double saldoAwal, String
pinAwal, double batasOverdraft) {
        // Memanggil konstruktor dari superclass (Rekening)
        super(nomor, nama, saldoAwal, pinAwal);
        this.batasOverdraft = batasOverdraft;
    }

    public double getBatasOverdraft() {
        return batasOverdraft;
    }

    // Menimpa (override) method tarik dari class induk khusus untuk Giro
    @Override
    public boolean tarik(double nominal) {
        if (nominal <= 0) {
            System.out.println("Nominal tarik harus lebih dari 0.");
            return false;
        }

        // Logika Overdraft: Boleh tarik melebihi saldo,
        // asalkan nominal tidak melebihi (saldo + batasOverdraft)
        if (nominal > (saldo + batasOverdraft)) {
            System.out.println("Gagal: Saldo dan limit overdraft tidak
cukup!");
            return false;
        }

        this.saldo -= nominal;
```

```
        String id = "TRX" + (riwayatTransaksi.size() + 1);
        riwayatTransaksi.add(new Transaksi(id, "Tarik Giro", nominal));
        System.out.println("Tarik Rp" + nominal + " berhasil. Saldo sekarang:
Rp" + saldo);
        return true;
    }
}
```

2.5.2 Penjelasan

- batasOverdraft: Batas maksimum pinjaman/dana cadangan yang diizinkan untuk ditarik ketika saldo utama habis.

- tarik(double nominal) (Polimorfisme / *Method Overriding*): Menimpa logika pencairan pada kelas induk. Penarikan tetap diizinkan meskipun melebihi jumlah saldo, selama total penarikan tidak melebihi saldo + batasOverdraft. Jika berhasil, saldo bisa bernilai negatif hingga batas *overdraft* yang ditentukan.

4.5 Program RekeningTabungan

4.5.1 Kode Program RekeningGiro.java

```
package pekan4;

public class RekeningTabungan extends Rekening {
    private double sukuBunga;

    public RekeningTabungan(String nomor, String nama, double saldoAwal, String
pinAwal, double sukuBunga) {
        super(nomor, nama, saldoAwal, pinAwal);
        this.sukuBunga = sukuBunga;
    }

    public void tambahBungaAkhirBulan() {
        double nominalBunga = saldo * (sukuBunga / 100);
        saldo += nominalBunga;

        String idTrx = "TRX-B-" + System.currentTimeMillis();
        riwayatTransaksi.add(new Transaksi(idTrx, "Bunga Bulanan",
nominalBunga));
```

```
        System.out.println("Bunga " + sukuBunga + "% berhasil ditambahkan: Rp"
+ nominalBunga);
    }

    public void tambahBungaAkhirTahun() {
        if (saldo > 10000000) {
            double nominalBunga = saldo * (sukuBunga / 100);
            saldo += nominalBunga;

            String idTrx = "TRX-T-" + System.currentTimeMillis();
            riwayatTransaksi.add(new Transaksi(idTrx, "Bunga Akhir Tahun",
nominalBunga));

            System.out.println("Selamat! Anda mendapat bonus bunga akhir tahun
" + sukuBunga + "%: Rp" + nominalBunga);
            System.out.println("Saldo Anda sekarang: Rp" + saldo);
        } else {
            System.out.println("Gagal: Saldo Anda (Rp" + saldo + ") tidak
memenuhi syarat minimum > Rp10.000.000 untuk bunga akhir tahun.");
        }
    }
}
```

4.5.2 Penjelasan

- sukuBunga: Menyimpan persentase bunga bulanan/tahunan.
- tambahBungaAkhirBulan(): Menghitung bunga berdasarkan saldo saat ini dan langsung menambahkan nominal bunga tersebut ke saldo serta riwayat transaksi.
- tambahBungaAkhirTahun(): Memberikan bonus bunga akhir tahun khusus bagi nasabah yang memiliki saldo di atas Rp10.000.000. Jika saldo tidak mencukupi syarat, penambahan bunga akan ditolak.

4.6 Output Tugas

1 Setelah user menginputkan Saldo Awal dan PIN, berikan opsi produk: "Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis"

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Simulasi Akhir Tahun (Khusus Tabungan)
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 123
Masukkan Nama Pemilik: nafi
Masukkan Saldo Awal: 50000
Masukkan PIN: 123456
Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis
Pilihan: 1
Masukkan Suku Bunga (%): 4
Rekening atas nama nafi berhasil dibuat.
```

2 Jika user memilih Tabungan, minta input sukuBunga dalam persen, lalu instansiasi objek RekeningTabungan.

```
Masukkan Suku Bunga (%): 4
Rekening atas nama nafi berhasil dibuat.
```

3 Jika user memilih Giro, minta input batasOverdraft (limit pinjaman), lalu instansiasi objek RekeningGiro.

```
Pilih menu: 1
Masukkan No Rekening: 456
Masukkan Nama Pemilik: ghifari
Masukkan Saldo Awal: 50000
Masukkan PIN: 123456
Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis
Pilihan: 2
Masukkan Batas Overdraft: 55000
Rekening atas nama ghifari berhasil dibuat.
```

4 Simpan objek tersebut ke dalam daftarRekening (Upcasting: Subclass otomatis dikenali sebagai tipe Superclass-nya).

```
5                       daftarRekening.add(rekeningBaru);
6                       akunAktif = rekeningBaru;
7                       break;
```

5 Tambahkan menu baru: "**7. Simulasi Akhir Bulan (Khusus Tabungan)**" di CLI.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Simulasi Akhir Tahun (Khusus Tabungan)
0. Keluar
Pilih menu: 7
Bunga 4.0% berhasil ditambahkan: Rp418080.0
```

6 Di dalam case 7:, periksa apakah akunAktif saat ini adalah objek Tabungan menggunakan operator ***instanceof***.

```
7    else if (akunAktif instanceof RekeningTabungan) {
8            RekeningTabungan tab = (RekeningTabungan) akunAktif;
9            tab.tambahBungaAkhirBulan();
10          } else {
11           System.out.println("Gagal: Fitur bunga akhir bulan hanya berlaku
     untuk Rekening Tabungan.");
```

7 Jika akun tersebut benar RekeningTabungan, lakukan Downcasting ke tipe Tabungan, lalu panggil method tambahBungaAkhirBulan().

```
8        public void tambahBungaAkhirTahun() {
9            if (saldo > 10000000) {
10               double nominalBunga = saldo * (sukuBunga / 100);
11               saldo += nominalBunga;
12
13               String idTrx = "TRX-T-" + System.currentTimeMillis();
14               riwayatTransaksi.add(new Transaksi(idTrx, "Bunga Akhir Tahun",
     nominalBunga));
15
16               System.out.println("Selamat! Anda mendapat bonus bunga akhir
     tahun " + sukuBunga + "%: Rp" + nominalBunga);
17               System.out.println("Saldo Anda sekarang: Rp" + saldo);
18           } else {
19               System.out.println("Gagal: Saldo Anda (Rp" + saldo + ") tidak
     memenuhi syarat minimum > Rp10.000.000 untuk bunga akhir tahun.");
20           }
21       }
```

8 Jika bukan, tolak dengan pesan: "Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan."

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Simulasi Akhir Tahun (Khusus Tabungan)
0. Keluar
Pilih menu: 7
Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.
```

PERTEMUAN 5

5.1 Program Main.java

5.1.1 Kode Program Main.java

```
package pekan5;

import java.util.ArrayList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        ArrayList<Rekening> daftarRekening = new ArrayList<>();
        Rekening akunAktif = null;
        boolean isRunning = true;

        System.out.println("=== SISTEM PERBANKAN MINI ===");

        while (isRunning) {
            System.out.println("\nMenu Utama:");
            System.out.println("1. Buka Rekening Baru");
            System.out.println("2. Setor Tunai");
            System.out.println("3. Tarik Tunai");
            System.out.println("4. Cek Informasi Rekening");
            System.out.println("5. Ganti Akun");
            System.out.println("7. Simulasi Akhir Bulan (Tabungan)");
            System.out.println("8. Transfer ke Rekening Lain (Tugas Modul
5)");
```

```
        System.out.println("0. Keluar");
        System.out.print("Pilih menu: ");

        int pilihan = input.nextInt();
        input.nextLine();

        switch (pilihan) {
            case 1:
                System.out.print("Masukkan No Rekening: ");
                String no = input.nextLine();
                System.out.print("Masukkan Nama Pemilik: ");
                String nama = input.nextLine();
                System.out.print("Masukkan Saldo Awal: ");
                double saldoAwal = input.nextDouble();
                input.nextLine();
                System.out.print("Masukkan PIN (6 digit): ");
                String pinAwal = input.nextLine();

                System.out.println("Pilih Produk: 1. Tabungan Umum | 2. Giro
Bisnis");
                int produk = input.nextInt();
                input.nextLine();

                Rekening rekeningBaru;
                if (produk == 1) {
                    System.out.print("Masukkan Suku Bunga (%): ");
                    double sukuBunga = input.nextDouble();
                    input.nextLine();
                    rekeningBaru = new RekeningTabungan(no, nama,
saldoAwal, pinAwal, sukuBunga);
                } else {
                    System.out.print("Masukkan Batas Overdraft: ");
                    double overdraft = input.nextDouble();
                    input.nextLine();
                    rekeningBaru = new RekeningGiro(no, nama, saldoAwal,
pinAwal, overdraft);
                }
                daftarRekening.add(rekeningBaru);
                akunAktif = rekeningBaru;
                break;
```

```
            case 2:
                if (akunAktif == null) System.out.println("Error: Belum ada
rekening aktif!");
                else {
                    System.out.print("Masukkan nominal setor: ");
                    akunAktif.setorTunai(input.nextDouble());
                    input.nextLine();
                }
                break;

            case 3:
                if (akunAktif == null) System.out.println("Error: Belum ada
rekening aktif!");
                else {
                    System.out.print("Masukkan nominal tarik: ");
                    akunAktif.tarikTunai(input.nextDouble());
                    input.nextLine();
                }
                break;

            case 4:
                if (akunAktif == null) System.out.println("Error: Belum ada
rekening aktif!");
                else akunAktif.cetakInfo();
                break;

            case 5:
                System.out.print("Masukkan nomor rekening: ");
                String nomorCari = input.nextLine();
                boolean ditemukan = false;
                for (Rekening r : daftarRekening) {
                    if (r.getNomorRekening().equals(nomorCari)) {
                        akunAktif = r;
                        ditemukan = true;
                        System.out.println("Berhasil mengganti akun ke " +
akunAktif.getNamaPemilik());
                        break;
                    }
                }
```

```
                if (!ditemukan) System.out.println("Rekening tidak
ditemukan!");
                break;

            case 7:
                if (akunAktif instanceof RekeningTabungan) {
                    ((RekeningTabungan)
akunAktif).tambahBungaAkhirBulan();
                } else {
                    System.out.println("Fitur khusus Rekening Tabungan.");
                }
                break;

            case 8:
                if (akunAktif == null) {
                    System.out.println("Error: Anda belum login/memiliki
rekening aktif!");
                    break;
                }
                System.out.print("Masukkan Nomor Rekening Tujuan: ");
                String noRekTujuan = input.nextLine();

                Rekening akunTujuan = null;
                for (Rekening r : daftarRekening) {
                    if (r.getNomorRekening().equals(noRekTujuan)) {
                        akunTujuan = r;
                        break;
                    }
                }

                if (akunTujuan == null) {
                    System.out.println("Rekening tujuan tidak ditemukan!");
                } else {
                    System.out.print("Masukkan Nominal Transfer: Rp");
                    double nominalTrf = input.nextDouble();
                    input.nextLine();
                    System.out.print("Masukkan PIN Akun Anda: ");
                    String pinTrf = input.nextLine();

                    akunAktif.transfer(akunTujuan, nominalTrf, pinTrf);
```

```
                }
                break;

            case 0:
                isRunning = false;
                System.out.println("Sistem ditutup. Terima Kasih!");
                break;

            default:
                System.out.println("Pilihan menu tidak tersedia!");
                break;
        }
    }
    input.close();
  }
}
```

5.1.2 Penjelasan

- Penyimpanan Objek Polimorfik dalam Struktur Koleksi (ArrayList<Rekening>): Program utama memanfaatkan struktur data ArrayList bertipe kelas induk (Rekening) untuk menampung berbagai macam objek turunan (RekeningTabungan dan RekeningGiro) secara bersamaan dalam satu daftar tunggal.

- Alur Interaksi Menu Utama & Manajemen Sesi (akunAktif): Menyediakan antarmuka berbasis teks interaktif dengan looping while untuk menangani pembukaan rekening baru, login akun, pengecekan saldo, setoran, hingga penarikan tunai.

- Integrasi Menu 8 untuk Eksekusi Transfer Antar Rekening: Menangkap input dari pengguna berupa nomor rekening tujuan, mencari objek yang sesuai di dalam daftarRekening secara iteratif, dan memanggil fungsi transfer() milik akunAktif untuk menyelesaikan transaksi secara aman dan dinamis.

5.2 Program Rekening.java

5.2.1 Kode Program Rekening.java

```
package pekan5;

import java.util.ArrayList;

public class Rekening {
   private String nomorRekening;
   private String namaPemilik;
   private String pin;

   protected double saldo;
   protected ArrayList<Transaksi> riwayatTransaksi;

   public Rekening(String nomor, String nama, double saldoAwal, String
pinAwal) {
      this.nomorRekening = nomor;
      this.namaPemilik = nama;
      this.saldo = saldoAwal;

      if (pinAwal.length() == 6) {
         this.pin = pinAwal;
      } else {
         System.out.println("Peringatan: PIN harus 6 digit! Menggunakan
PIN default 123456");
         this.pin = "123456";
      }
      this.riwayatTransaksi = new ArrayList<>();
      System.out.println("Rekening atas nama " + namaPemilik + " berhasil
dibuat.");
   }

   public String getNomorRekening() { return nomorRekening; }
   public String getNamaPemilik() { return namaPemilik; }
   public double getSaldo() { return saldo; }
   public ArrayList<Transaksi> getRiwayatTransaksi() { return
riwayatTransaksi; }
```

```
    public boolean otentikasi(String inputPin) {
        return this.pin.equals(inputPin);
    }

    public boolean setorTunai(double nominal) {
        if (nominal <= 0) {
            System.out.println("Nominal setor harus lebih dari 0.");
            return false;
        }
        this.saldo += nominal;
        String id = "TRX" + (riwayatTransaksi.size() + 1);
        riwayatTransaksi.add(new Transaksi(id, "Setor Tunai", nominal));
        System.out.println("Setor Rp" + nominal + " berhasil. Saldo sekarang:
Rp" + saldo);
        return true;
    }

    public boolean tarikTunai(double nominal) {
        if (nominal <= 0) {
            System.out.println("Gagal: Nominal tarik harus lebih dari 0!");
            return false;
        }
        if (nominal > saldo) {
            System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo
Anda: Rp" + saldo);
            return false;
        }
        this.saldo -= nominal;
        String id = "TRX" + (riwayatTransaksi.size() + 1);
        riwayatTransaksi.add(new Transaksi(id, "Tarik Tunai", nominal));
        System.out.println("Tarik tunai Rp" + nominal + " berhasil. Sisa
saldo: Rp" + saldo);
        return true;
    }

    public void transfer(Rekening tujuan, double nominal, String pinInput) {
        if (!this.otentikasi(pinInput)) {
            System.out.println("Transfer Gagal: PIN salah!");
            return;
        }
```

```
        if (this.nomorRekening.equals(tujuan.getNomorRekening())) {
            System.out.println("Transfer Gagal: Tidak dapat mentransfer ke
rekening sendiri!");
            return;
        }

        System.out.println("Memproses transfer sebesar Rp" + nominal + " ke
" + tujuan.getNamaPemilik() + " (" + tujuan.getNomorRekening() + ")...");

        boolean statusTarik = this.tarikTunai(nominal);

        if (statusTarik) {
            tujuan.setorTunai(nominal);

            String idSender = "TRX" + (riwayatTransaksi.size() + 1);
            riwayatTransaksi.add(new Transaksi(idSender, "Transfer Keluar ke
" + tujuan.getNomorRekening(), nominal));

            String idReceiver = "TRX" + (tujuan.getRiwayatTransaksi().size() +
1);
            tujuan.getRiwayatTransaksi().add(new Transaksi(idReceiver,
"Transfer Masuk dari " + this.nomorRekening, nominal));

            System.out.println("Transfer BERHASIL dilakukan!");
        } else {
            System.out.println("Transfer Gagal diproses.");
        }
    }

    public void cetakInfo() {
        System.out.println("No. Rekening : " + nomorRekening);
        System.out.println("Pemilik      : " + namaPemilik);
        System.out.println("Saldo        : Rp" + saldo);
    }
}
```

5.2.2 Penjelasan

- Implementasi Dynamic Method Dispatch pada tarikTunai(): Method tarikTunai(double nominal) di kelas induk ini mengembalikan nilai boolean dan berfungsi sebagai kerangka dasar penarikan saldo normal. Ketika dipanggil melalui objek turunan (seperti RekeningGiro), Java secara dinamis mengeksekusi versi method yang sesuai saat runtime.

- Perancangan Argumen Polimorfik pada Method transfer(): Method transfer(Rekening tujuan, double nominal, String pinInput) dirancang dengan argumen bertipe kelas induk (Rekening tujuan). Hal ini memungkinkan method menerima objek apa saja dari kelas anak (baik RekeningTabungan maupun RekeningGiro) secara fleksibel tanpa perlu membuat banyak method transfer yang terpisah.

- Validasi Keamanan dan Integritas Transaksi: Di dalam method transfer(), sistem melakukan verifikasi PIN (otentikasi) dan pencegahan transfer ke nomor rekening sendiri sebelum memicu proses penarikan dana dan penyetoran ke rekening tujuan secara otomatis.

5.3 Program Transaksi.java

5.3.1 Kode Program Transaksi.java

```
package pekan5;

public class Transaksi {
    private String idTransaksi;
    private String jenisTransaksi;
    private double nominal;

    public Transaksi(String idTransaksi, String jenisTransaksi, double
nominal) {
        this.idTransaksi = idTransaksi;
        this.jenisTransaksi = jenisTransaksi;
        this.nominal = nominal;
    }

    @Override
```

```
    public String toString() {
        return "[" + idTransaksi + "] " + jenisTransaksi + " sebesar Rp" +
nominal;
    }
}
```

5.3.2 Penjelasan

- Inisialisasi Objek Data Transaksi: Kelas ini bertindak sebagai blueprint untuk menyimpan atribut spesifik setiap transaksi yang dilakukan pengguna, meliputi idTransaksi, jenisTransaksi (seperti "Setor Tunai", "Tarik Tunai", atau "Transfer"), dan jumlah nominal.

- Penggunaan Konstruktor Berparameter: Konstruktor Transaksi(...) dipanggil setiap kali terjadi aktivitas keuangan untuk langsung mengisi data transaksi dan mengikatnya ke dalam riwayat masing-masing rekening.

- Overriding Method toString(): Kelas ini menimpa (@Override) bawaan dari objek dasar Java (Object) agar ketika objek Transaksi dipanggil ke dalam sistem cetak, ia langsung mengembalikan format teks yang rapi dan mudah dibaca ([id] jenis sebesar Rp...).

5.4 Program RekeningGiro

5.4.1 Kode Program RekeningGiro.java

```
package pekan5;

public class RekeningGiro extends Rekening {
    private double batasOverdraft;

    public RekeningGiro(String nomor, String nama, double saldoAwal,
String pinAwal, double batasOverdraft) {
        super(nomor, nama, saldoAwal, pinAwal);
        this.batasOverdraft = batasOverdraft;
    }
```

```
    public double getBatasOverdraft() { return batasOverdraft; }

    @Override
    public boolean tarikTunai(double nominal) {
        if (nominal <= 0) {
            System.out.println("Gagal: Nominal tarik Giro harus lebih dari 0!");
            return false;
        }
        if (nominal > (saldo + batasOverdraft)) {
            System.out.println("Transaksi Gagal: Melebihi batas overdraft!
Maksimal penarikan: Rp" + (saldo + batasOverdraft));
            return false;
        }

        this.saldo -= nominal;
        String id = "TRX" + (riwayatTransaksi.size() + 1);
        riwayatTransaksi.add(new Transaksi(id, "Tarik Giro (Overdraft)",
nominal));
        System.out.println("Tarik tunai Giro Rp" + nominal + " berhasil.
Saldo saat ini: Rp" + saldo);
        return true;
    }
}
```

5.4.2 Penjelasan

- Penerapan Pewarisan (extends) dan Konstruktor super: Kelas ini mewarisi seluruh atribut dan method dari kelas Rekening dengan menggunakan kata kunci extends, serta meneruskan parameter inisialisasi dasar ke kelas induk menggunakan fungsi super(...).

- Penambahan Atribut Khusus batasOverdraft: Kelas anak ini memperluas fungsionalitas perbankan dengan menambahkan properti khusus bisnis giro, yaitu batas pinjaman sementara (overdraft) di luar saldo riil pemilik akun.

- Penerapan Method Overriding (@Override) pada tarikTunai(): Kelas ini menimpa method penarikan dari kelas induk (tarikTunai) untuk mengubah aturan

bisnisnya. Aturan baru mengizinkan penarikan melebihi saldo utama asalkan total penarikan tidak melampaui (saldo + batasOverdraft).

5.5 Program RekeningTabungan

5.5.1 Kode Program RekeningTabungan.java

```
package pekan5;

public class RekeningTabungan extends Rekening {
   private double sukuBunga;

   public RekeningTabungan(String nomor, String nama, double saldoAwal,
String pinAwal, double sukuBunga) {
      super(nomor, nama, saldoAwal, pinAwal);
      this.sukuBunga = sukuBunga;
   }

   public void tambahBungaAkhirBulan() {
      double nominalBunga = saldo * (sukuBunga / 100);
      saldo += nominalBunga;
      String idTrx = "TRX" + (riwayatTransaksi.size() + 1);
      riwayatTransaksi.add(new Transaksi(idTrx, "Bunga Bulanan",
nominalBunga));
      System.out.println("Bunga " + sukuBunga + "% berhasil
ditambahkan: Rp" + nominalBunga);
   }

   public void tambahBungaAkhirTahun() {
      if (saldo > 10000000) {
         double nominalBunga = saldo * (sukuBunga / 100);
         saldo += nominalBunga;
         String idTrx = "TRX" + (riwayatTransaksi.size() + 1);
         riwayatTransaksi.add(new Transaksi(idTrx, "Bunga Akhir Tahun",
nominalBunga));
         System.out.println("Selamat! Bonus bunga akhir tahun " +
sukuBunga + "%: Rp" + nominalBunga);
      } else {
```

```
            System.out.println("Gagal: Saldo minimum Rp10.000.000
diperlukan.");
        }
    }
}
```

5.5.2 Penjelasan

- Spesialisasi Fitur Produk Tabungan: Kelas ini diturunkan dari Rekening untuk menangani aturan bisnis khusus nasabah penabung, seperti pengelolaan persentase suku bunga harian/bulanan/tahunan.

- Perhitungan Otomatis Bunga Akhir Bulan: Memiliki method tambahBungaAkhirBulan() yang mengalikan saldo berjalan dengan besaran sukuBunga, lalu otomatis menambahkannya ke saldo dan mencatatnya ke dalam riwayatTransaksi.

- Validasi Syarat Minimum Saldo Akhir Tahun: Method tambahBungaAkhirTahun() menerapkan logika seleksi (if-else) untuk memastikan bonus bunga akhir tahun hanya diberikan kepada nasabah yang memiliki saldo di atas batas minimum (misalnya > Rp10.000.000).

5.6 Output

1. Pembuatan Rekening Baru (Menu 1 - Tabungan & Giro)

```
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
7. Simulasi Akhir Bulan (Tabungan)
8. Transfer ke Rekening Lain (Tugas Modul 5)
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 1
Masukkan Nama Pemilik: na
Masukkan Saldo Awal: 1000000
Masukkan PIN (6 digit): 111111
Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis
1
Masukkan Suku Bunga (%): 2
Rekening atas nama na berhasil dibuat.
```

```
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
7. Simulasi Akhir Bulan (Tabungan)
8. Transfer ke Rekening Lain (Tugas Modul 5)
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 2
Masukkan Nama Pemilik: fi
Masukkan Saldo Awal: 50000
Masukkan PIN (6 digit): 111111
Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis
2
Masukkan Batas Overdraft: 60000
Rekening atas nama fi berhasil dibuat.
```

2. Setor Tunai (Menu 2)

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
7. Simulasi Akhir Bulan (Tabungan)
8. Transfer ke Rekening Lain (Tugas Modul 5)
0. Keluar
Pilih menu: 2
Masukkan nominal setor: 50000
Setor Rp50000.0 berhasil. Saldo sekarang: Rp100000.0
```

3. Tarik Tunai Normal pada Rekening Tabungan (Menu 3)

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
7. Simulasi Akhir Bulan (Tabungan)
8. Transfer ke Rekening Lain (Tugas Modul 5)
0. Keluar
Pilih menu: 3
Masukkan nominal tarik: 200000
Transaksi Gagal: Melebihi batas overdraft! Maksimal penarikan: Rp160000.0
```

4. Tarik Tunai dengan Overdraft pada Rekening Giro (Menu 3 dengan Overriding)

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
7. Simulasi Akhir Bulan (Tabungan)
8. Transfer ke Rekening Lain (Tugas Modul 5)
0. Keluar
Pilih menu: 3
Masukkan nominal tarik: 150000
Tarik tunai Giro Rp150000.0 berhasil. Saldo saat ini: Rp-50000.0
```

5. Simulasi Bunga Akhir Bulan / Akhir Tahun (Menu 7)

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
7. Simulasi Akhir Bulan (Tabungan)
8. Transfer ke Rekening Lain (Tugas Modul 5)
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 1
Masukkan Nama Pemilik: annafi
Masukkan Saldo Awal: 100000
Masukkan PIN (6 digit): 111111
Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis
1
Masukkan Suku Bunga (%): 2
Rekening atas nama annafi berhasil dibuat.
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
7. Simulasi Akhir Bulan (Tabungan)
8. Transfer ke Rekening Lain (Tugas Modul 5)
0. Keluar
Pilih menu: 7
Bunga 2.0% berhasil ditambahkan: Rp2000.0
```

6. Transfer Antar Rekening (Menu 8 - Tugas Utama Modul 5)

- Sangat disarankan mengambil 2 kondisi:

- Kondisi Berhasil: Memasukkan nomor rekening tujuan yang valid, nominal transfer, dan PIN yang benar, lalu muncul pesan transfer berhasil serta saldo pengirim berkurang dan saldo penerima bertambah.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
7. Simulasi Akhir Bulan (Tabungan)
8. Transfer ke Rekening Lain (Tugas Modul 5)
0. Keluar
Pilih menu: 8
Masukkan Nomor Rekening Tujuan: 1
Masukkan Nominal Transfer: Rp20000
Masukkan PIN Akun Anda: 111111
Memproses transfer sebesar Rp20000.0 ke annafi (1)...
Tarik tunai Rp20000.0 berhasil. Sisa saldo: Rp30000.0
Setor Rp20000.0 berhasil. Saldo sekarang: Rp122000.0
Transfer BERHASIL dilakukan!
```

- Kondisi Gagal: Mencoba transfer ke rekening sendiri, salah PIN, atau saldo tidak mencukupi untuk memperlihatkan sistem validasi error.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
7. Simulasi Akhir Bulan (Tabungan)
8. Transfer ke Rekening Lain (Tugas Modul 5)
0. Keluar
Pilih menu: 8
Masukkan Nomor Rekening Tujuan: 8
Rekening tujuan tidak ditemukan!
```

KESIMPULAN

Penerapan Konsep Polimorfisme (Runtime Polymorphism): Praktikum ini membuktikan bahwa polimorfisme memungkinkan pemanggilan method tarikTunai() dieksekusi secara dinamis (Dynamic Method Dispatch / Late Binding) sesuai dengan tipe objek aslinya pada saat runtime (apakah itu objek dari kelas RekeningTabungan atau RekeningGiro), tanpa memerlukan percabangan if-else yang rumit di dalam kode utama.

Implementasi Method Overriding: Penggunaan anotasi @Override pada kelas RekeningGiro memudahkan modifikasi aturan bisnis penarikan tunai — di mana akun giro diizinkan melakukan penarikan hingga melewati saldo utama dengan batas toleransi pinjaman sementara (overdraft) — tanpa merusak struktur kode yang sudah ada pada kelas induk (Rekening).

Argumen Polimorfik pada Fitur Transfer: Perancangan method transfer(Rekening tujuan, double nominal, String pinInput) dengan parameter bertipe kelas induk (Rekening) menunjukkan fleksibilitas tinggi dalam PBO. Sistem dapat menangani transfer antar berbagai jenis rekening (Tabungan ke Giro, Giro ke Tabungan, atau antar Tabungan) secara aman, modular, dan mudah dikembangkan untuk jenis produk perbankan lainnya di masa depan.